check_cm_lan_routes_authenticated

check_cm_lan_routes_authenticated.sh

Revision: 10 **Last modified:** 2026-08-27T01:32:30Z **Review round:** 13 (independent Fable review, §11.4.209 → §11.4.134) **Gate:** `CM-LAN-ROUTES-AUTHENTICATED` **Authority:** operator decision BOB-102 (2026-08-26) · §11.4.135 · §11.4.201

Overview

On 2026-08-26 the operator decided BOB-102 as **"Keep 0.0.0.0 + auth guard"**. The tunnel stays LAN-reachable — no bind address moves to `127.0.0.1` — and that exposure was accepted **on the condition** that a permanent regression guard fails the build if any LAN-reachable route ever stops demanding authentication. This gate is that condition made mechanical.

It enumerates every route served by a listener declared LAN-bound **through the registration idioms it models**, resolves per route whether authentication genuinely reaches it, and refuses the build when such a route is neither auth-wired nor covered by a justified, checked-in exemption. Registration forms it does *not* model fail closed rather than passing silently — see Modelled scope.

Prerequisites

- `bash`, `python3` (>= 3.9), and `PyYAML`.
- The sibling analyzer engine `scripts/pre_build/lan_route_auth_analyzer.py`.
- The policy file `config/lan_route_auth_policy.yaml`.

No container, no running service, and no network access is required — the gate is pure static analysis.

Usage

```
./scripts/pre_build/
        check_cm_lan_routes_authenticated.sh            # default
        scope
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --
        list     # JSON inventory
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --policy
        P --root R
./scripts/pre_build/check_cm_lan_routes_authenticated.sh --help
```

Exit	Meaning
`0`	PASS — every LAN-reachable route is auth-wired or justly exempted
`1`	FAIL — one or more findings, each printed with resolved evidence
`2`	ERROR — usage error, missing analyzer/policy, or no usable `python3`

Why it does not grep for “auth”

A gate that greps for a token *mentioning* auth asserts a **proxy signal**, not the real condition (§11.4.201). A comment, a variable name or a log string would satisfy it while the route stayed wide open. This gate resolves the real wiring from the authoritative source, per framework:

Service kind	A route counts as protected when…
`fastapi`	the decorator verb is one of GET/POST/PUT/DELETE/ PATCH/HEAD/OPTIONS/ **TRACE** (checked against the installed FastAPI source, not from memory), the module parses with `ast`, and a `Depends(<marker>)` genuinely reaches the route — as a handler parameter default, as the decorator’s `dependencies=[...]`, or via its `APIRouter(dependencies=[...])`.
`gin`	an auth marker is installed with `.Use(...)` on that **exact** owner, or inherited down the

Service kind	A route counts as protected when…
	`Group(...)` chain. gin's **registration-order** semantics are honoured: a route registered *before* `.Use` on its owner is **not** covered by it, and a group inherits only what its parent carried at `Group()` time.
`gomux`	the **specific** `http.NewServeMux()` value the route was registered on is the one an auth marker wraps in a `return`. Resolution is **per mux variable**, never per file.

Per-mux wrap resolution

A whole-file "does an auth wrap appear anywhere" test would mark a second, bare mux as protected just because a *different* mux in the same file is wrapped — a **false attribution of safety**, which is categorically worse than failing to see a route at all. This gate resolves the wrap for the exact mux variable a route was registered on, following assignment indirection to a fixpoint, and treats an unresolvable wrap as **unwrapped** (fail closed).

> **The fixpoint's own boundary, stated (round 7).** Revision 6 described this resolution as fail-closed and stopped there. That is true of the direction it was describing — a carrier the pass *misses* only shrinks the name set, which can move a verdict toward UNWRAPPED and never toward WRAPPED — and **false of the other direction**, which was not recorded. The propagation is purely lexical: any assignment whose right-hand side *mentions* a carrier name adopts its left-hand side, with no model of what that expression returns. So `stats := describe(mux)` followed by `return WithAuth(stats)` credits `mux` (measured `auth_wired=true`) although nothing wrapping `mux` was ever returned. This is an **over-approximation, and over-approximating the carrier set is fail-open.** It is kept deliberately — the alternative is a Go return-value model, a second parser with its own failure modes standing between this gate and the one question it answers — and it is bounded by two rules that do hold: only *function-level* returns count, and both ends of the name match are anchored, so neither a closure's return nor a same-prefix or field-qualified name can carry a credit. It is now owned in `_carriers`, in the skip table, and pinned by the `gomux-carrier-over-approximates` fixture, so

> narrowing it later is a deliberate change rather than drift. It is carried as an **honest open gap** (§11.4.118), not as a solved problem.

Only **function-level** returns count, and they are read from a projection of the source in which comments and *every* literal have been blanked by a single stateful pass over Go's lexical modes — `code`, `//` line comment, `/* */` block comment, backtick raw string, `"` interpreted string, `'` rune — with the mode carried **across line boundaries**, because a block comment and a raw string both span lines. Three shapes that must never prove authentication, and do not:

```
//
        TODO: return WithAuth(mux) once BOB-197 lands   <- a
        comment is a CARRIER
return mux                                              //
        (§11.4.201(7)(a))

makeAuthed := func() http.Handler { return WithAuth(mux) }  //
        closure return
return mux                                                  //
        belongs to the closure

const usage = `the authenticated variant would
  return WithAuth(mux)          <- a MULTI-LINE raw string is a
        CARRIER too
`
return mux
```

All three are reported UNAUTHENTICATED. Handler closures passed to `HandleFunc` — the ordinary shape in this repo's own router — are unaffected, because only `return`s inside a closure *body* are excluded, not the enclosing function's.

The mode that wins is the one whose opener comes **first**: a `/*` written inside a `//` comment is comment text (it does not open a block comment, and must not blank the real code below it — that would be a §11.4.201(1) FAIL-bluff, and a gate that cries wolf gets bypassed); a backtick inside a comment does not open a raw string; `//` and `/*` inside a raw string are string content; escapes apply inside `"` and `'` but **not** inside a raw string, where a trailing `\` is a literal backslash.

One rule spans both Go extractors: **a construct counts only when its code skeleton survives in the literal-free projection.** The `code` projection keeps interpreted-string literals, because every route regex reads its path out of a `"..."` — so scanning `code` for a *call* would let a literal carry the call. A single log line reading `log.Println("hint: r.Use(middleware.AuthMW())")` would otherwise mark every route on `r` as covered by middleware that does not exist. The registration call, the engine declaration and the `.Use` are therefore read from the literal-free projection;

only the **path text** is read from the literal. The same rule runs in the other direction: a doc string that merely *mentions* `.Any(` must not mint a refusal, or the gate cries wolf and gets bypassed (§11.4.201(1)). A route path written as a raw string is not resolvable by those regexes, so it surfaces as an unmodelled registration — refused, never silently dropped from the scan.

A construct that cannot be lexed to a close — an unterminated `/*` or backtick at end of file, or an unterminated `"/'` at end of line, none of which is valid Go — is **refused** with its `file:line`, never silently swallowed. The pre-round-4 engine failed *open* here: an unterminated `/*` matched no `/\*.*?\*/` span, so a fake wrap inside it survived and proved the wrap. Fail closed is the rule on exactly the input the engine cannot lex (§11.4.252).

Every refusal prints the route, its `file:line`, how the verdict was reached, and whether the method is mutating — so a false positive is diagnosable in one step rather than by re-deriving the gate's reasoning.

Modelled scope and fail-closed behaviour

This gate does **not** claim to see every route a framework can express. It claims to see the forms in the table above, and to **fail closed** on any it does not model:

- FastAPI `add_api_route` / `add_route` / `.mount` / `.websocket`
- FastAPI `@router.api_route(...)` — its path *is* a literal, but its method set comes from a `methods=[...]` keyword this model does not read, so the route cannot be resolved to a method (round 5)
- FastAPI `@router.route("/p", methods=[...])` — a real registration decorator (`routing.py:1317` in the installed FastAPI, 102 lines *above* the `api_route` round 5 found in the same file). It calls `add_route()`, which builds a plain starlette `Route` that FastAPI's dependency injection never runs on, so a `Depends(marker)` can never reach it. It is refused permanently rather than modelled (round 6)
- a FastAPI registration reached through a declared router but **not** through a `@router.verb("/literal")` decorator on a function — an immediate call (`router.post("/p")(fn)`), a decorator on a **class**, or a bound-method alias (`reg = router.post` then `@reg("/p")`). All three registered live routes and all three were silently dropped before round 6. The rule is keyed on the **owner being a declared router**, not on the verb name: `.get` is both an HTTP verb and the most common method name in Python, so an unqualified rule would fire on `d.get(k)` and every HTTP-client call in the tree (round 6)

- a FastAPI **router bound more than once** in a module — which router object a decorator on that name refers to is not statically decidable, so neither its prefix nor its router-level dependencies can be attributed to any particular route (round 6)
- a registration whose **receiver is separated from its member by whitespace or a newline**, in EITHER Go extractor — `r.` then `POST("/p", h)` on the next line, `open.` then `HandleFunc("/p", h)`, `r.  POST("/p", h)`, `mux . Handle("/p", h)`. Go permits arbitrary whitespace on either side of a selector dot and inserts no semicolon after a trailing one, so every spelling is legal and the split-at-newline form is **gofmt-stable** (verified with `gofmt -e`). Round 6 refused this for gin only; the gomux twin silently dropped the route until round 11 fixed the primitive for both (round 11)
- a **router-group declaration** whose parent is separated from `.Group(` the same way (`g := r.` then `Group("/admin")`). This one is refused rather than merely dropped because it corrupts the route **path**: every registration on that group loses its prefix, so `POST /admin/wipe` resolves as `POST /wipe` and an exemption written for a genuinely public `POST /wipe` matches it (round 11)
- a **gomux registration on a dotted receiver** (`s.mux.HandleFunc("/p", h)`), whose wrap cannot be followed — the round-5 gin defect living in the third extractor, found by round 6's credit audit (round 6)
- a FastAPI route registered on a **dotted or computed** router expression (`@<expr>.<attr>.post("/p")`), whose prefix and router-level dependencies cannot be resolved statically (round 5)
- a **gin registration sharing a line with another statement** — e.g. `g := r.Group("/api/v1"); g.POST("/wipe", h)` or `r.POST("/p", h); r.Use(authMW())`. gin middleware is order-dependent and this resolver's ordering model is line-granular, so on a multi-statement line neither the route's coverage nor a same-line `.Group()`/`.Use()` ordering is decidable. Detected on the comment-and-literal-blanked projection, so a `;` inside a path literal (legal — matrix parameters) or inside a trailing `//` comment never mints a refusal (round 5)
- a gin route registered on a receiver reached through a **field or selector** (`s.router.POST("/p", h)`), whose middleware chain cannot be followed — attributing it to a same-named local engine or group would falsely mark it authenticated (round 5)
- gin `.Any` / `.Match` / `.Handle` / `.Static*` / `.NoRoute` / `.NoMethod`
- a multi-line or computed (non-literal) path expression in any framework
- registration on the global `http.DefaultServeMux`
- a **gomux receiver this model cannot name** — a call result, an index expression or any other computed receiver (`newServeMux().HandleFunc(...)`, `s.buildMux().HandleFunc(...)`, `muxes[0].HandleFunc(...)`). **Known over-refusal (round 8, documented round 11):** a plain *call* to a method named `Handle` on such a receiver — `getHandler().Handle(w, r)` — is

statically indistinguishable from a registration and is refused too. This is not a new class: the shipped dotted-receiver rule already refuses the twin `x.y.Handle(w, r)` on the same reasoning, and gin's catch-all over-refuses identically. A dotted handler *argument* (`d.Health.HandleHealth`) is untouched, because the member must be followed by `(` to count and an argument is followed by `,` or `)`. See False positives are failures too

- more than one gin engine **declaration site** in a single file (cross-engine handler and middleware sharing is not modelled). Counted per site rather than per name since round 6: two functions each declaring `r := gin.New()` previously collapsed into one entry, so the refusal never fired and one function's `.Use` credited the other function's route
- a FastAPI guard alias that is **not** a single module-level binding — one assigned twice, or assigned inside a conditional (`if os.getenv("DISABLE_AUTH")`), since which dependency it carries at runtime is not statically decidable and trusting it would be a fail-**open** read. *Known limit, recorded where its symptom appears (round 7):* the alias maps are keyed on the module **basename**, so `a/util.py` and `b/util.py` collide into one entry. The collision is **measured fail-closed** — the merged entry trips this very rule and is refused with a finding, so no credit crosses a package boundary — but that is incidental rather than designed, and it means a correct alias can be refused because an unrelated package has a same-named module. Until round 7 this note lived only in the round-6 evidence file, which is not where a reader hitting the refusal will look
- a **gin group name bound more than once** in a file, or bound by a declaration this model cannot resolve (`g := s.pub.Group("/public")` — a parent reached through a selector, or a prefix that is not a string literal). `groups` was keyed on the name and scoped to the file, so a name the modelled pattern could not see was invisibly re-bound and the later registration inherited the earlier binding's **prefix and its coverage**: measured `POST /admin/wipe auth_wired=true`, findings 0, exit 0 on a route that is neither at that path nor authenticated. Counted per **declaration site** now, the same rule already applied to gin engines and FastAPI routers (round 7)
- a **gomux registration on a receiver reached through a selector**, in every spelling. The check keyed on an enumerated list of characters an expression may end with, and `}` was not in it, so `Server{}.mux.HandleFunc("/p", h)` matched neither that check nor the bare-owner path and was **silently dropped**. It now keys on the receiver being reached through a dot, which is the condition that was actually load-bearing (round 7)
- a **FastAPI registration reached through an imported router** — `from .routers import router` then `router.post("/wipe")(wipe)` in the importing module. The rule's owner test asked whether *this* module declared the router, so the whole rule was one `import` away from being bypassed and the mutating route vanished (findings 0, exit 0). The owner is now resolved

transitively through the import graph, and still has to resolve to a router some module declared — the qualification that keeps `d.get(k)` and every HTTP-client call out of it (round 7)

Each is reported as an `UNMODELLED REGISTRATION IDIOM` and **fails the gate**. It is never silently skipped. A guard whose stated purpose is catching tomorrow's drift must not answer "clean" about a construct it cannot see (§11.4.201(7)(c) — the path is part of the instrument). Note that the zero-routes FALSE-NULL check cannot catch this case, because the service still resolves its *other* routes and returns a confident, wrong count.

How that claim is kept honest. Defects kept reaching this gate in which a `continue` stepped over a route with *no* finding — the claim above was documented before it was earned. The analyzer's module docstring therefore enumerates **every skip in every extractor** and gives each one of three verdicts: `REFUSES` (emits a finding, gate fails), `FILTER` (proven not to be a registration — no route to lose), or `DROPS` (the defect class). The enumeration lives in the source so a reviewer can check it against the code rather than take this paragraph on faith (§11.4.250 — fix the primitive, not the third door).

> **Retraction (round 6, §11.4.118).** Revision 5 of this guide asserted that "the `DROPS` column is empty, and a future entry in it is a release blocker." **That claim is withdrawn.** It was never earnable by inspection — *absence of evidence of looking is not evidence of absence* — and three consecutive reviews falsified it by attacking a surface the previous round had not attacked; round 6 found four dropped FastAPI idioms in minutes, and a further four false-credit leaks, purely by looking somewhere new. The claim was also actively harmful: it promoted every idiom anyone might discover later into a standing release blocker, which is a promise this gate cannot keep and which penalises exactly the discovery that keeps it honest.
>
> What this gate claims instead is narrower and checkable:
>
> - **the enumerated set** — the skips listed in the analyzer docstring are the ones actually exercised, each with a fixture in the meta-test, and every `REFUSES` verdict pinned by a fixture that dies when the refusal is removed. Round 7 found that claim two rows short of true: **no fixture contained a `*_test.go` at all**, so the test-file `FILTER` in *both* Go extractors was unexercised in either direction. `gin-test-file-skipped` and `gomux-test-file-skipped` close it — each carries an unauthenticated mutating route inside a test file, and each turns red the moment the filter stops filtering. That last part was *measured*, not assumed: writing this retraction raised the question, so every refusal was deleted one at a time in a scratch copy and

> the harness re-run. Six survived — `websocket_route`, gin `.Match`, gin `.NoRoute`, `http.DefaultServeMux`, an unknown service kind, and an exemption missing its `path`. All six are now pinned, and each new pin was itself checked to fail under the deletion;
> - **the method** — those surfaces were derived by reading the *installed* framework source (§11.4.201, never memory), by walking the constructs this engine uses to mark a route protected against the credit rule, and by reproducing each candidate against the shipped analyzer before calling it a defect (§11.4.199). A stated method over an enumerated set — round 7 corrected the earlier wording ("auditing *every* construct"), which was the same completeness assertion this retraction withdraws, moved one column over;
> - **the honest gap** — an idiom outside that set is **unsurveyed, not proven absent**. A newly-found one is a tracked coverage gap to be closed with a fixture and a coverage-escape note (§11.4.238), *not* a broken promise.
>
> The `DROPS` column stays, and stays filled in honestly. Only the completeness assertion is gone.

The credit-bearing primitive (round 6, corrected round 7). Round 5 fixed one dotted-receiver leak because that was the one reported. Round 6's review named the real rule: a **credit** — anything that marks a route protected — must never be attributed across a scope boundary the resolver cannot see. That rule was then walked across the credit-bearing constructs this engine uses, and four leaked. Each was a *false attribution of safety*, which this gate treats as categorically worse than a missed route: the gate does not merely fail to see the route, it asserts the route is protected.

> **Retraction, second column (round 7, §11.4.118).** Revision 6 retracted the `DROPS`-completeness claim and then wrote a fresh absolute one column over: *"every construct able to mark a route protected was audited; the four that leaked are pinned."* **That claim is withdrawn too.** Round 7 falsified it twice, in two constructs the same audit had recorded as *"measured fail-closed"* — a gin group name re-bound by an unresolvable declaration, and a `http.NewServeMux()` declaration reached through a selector. Both returned `auth_wired=true`, findings 0, exit 0 on an unauthenticated mutating route. "Audited" is a record of where somebody **looked**; inspection cannot convert that into "there are no others", and this engine had to learn the same lesson a second time in the neighbouring column. What is claimed for the credit side is now exactly what is claimed for the drop side — an **enumerated set**, a **stated method**, and an **honest gap** — and each per-construct verdict in the analyzer is a **dated measurement**: "fail-closed" means

> *measured fail-closed on the shapes tried*, never *cannot leak*. A credit construct found later is a tracked coverage escape (§11.4.238), not a broken promise.

leak	shape	pre-fix verdict
gin `.Use` on a dotted receiver	`s.router.Use(auth)` credited a local `router`	mutating route `auth_wired=true`, exit 0
gomux dotted registration	`s.mux.HandleFunc(...)` inherited a local mux's wrap	mutating route `auth_wired=true`, exit 0
two gin engines sharing a name	two functions' `r := gin.New()` collapsed to one entry	one function's `.Use` credited the other's route
re-assigned FastAPI router	a later `APIRouter(dependencies=[…])` back-credited earlier routes	route on the *unguarded* router `auth_wired=true`
gin group name re-bound (round 7)	`g := r.Group("/admin")` in one function, `g := s.pub.Group("/public")` in another	`POST /admin/wipe` `auth_wired=true`, exit 0 — wrong path *and* falsely safe
gomux mux declared through a selector (round 7)	`s.mux = http.NewServeMux()` declared a bare-name mux that does not exist	mutating route on a *different* local `mux` `auth_wired=true`, exit 0
FastAPI router reached by import (round 7)	`from .routers import router` then `router.post("/wipe")(wipe)`	the mutating route vanished entirely — findings 0, exit 0

To ship a route in an unmodelled form, either re-express it in a modelled one or extend the analyzer to model it — and add a fixture so it stays guarded.

False positives are failures too

A refusal on a genuinely public route is a FAIL-bluff of the same severity as a missed route (§11.4.201(1)). Deliberately-public routes are therefore declarable — but **never silently**. Every exemption entry requires:

- `class:` — either `public-by-design` or `known-gap`
- `justification:` — a non-empty explanation

The gate refuses an entry missing either, so a blanket skip is impossible.

The same discipline applies to the policy's own emptiness (round 5): a policy declaring **zero services** is refused, because an empty declaration list and a system with no LAN listener return the identical quiet PASS (§11.4.201(6) FALSE-NULL) — deleting the declarations must not be a way to turn the gate green. A listener that is genuinely not LAN-reachable is declared with `lan_bound: false`, which remains a tested, accepted scoping statement.

`known-gap` entries are **real holes carried honestly**, not endorsements. The gate prints their count loudly on every run. They are expected to shrink and never grow (§11.4.261).

The drift property (why the route set is derived, never listed)

The inventory is re-derived from source on every run. A hand-maintained list drifts silently, and silent drift is exactly what this guard exists to stop. The consequence is the useful one:

- A **new route** is in neither the auth-wired set nor the exemption list, so it FAILS until somebody consciously protects or justifies it.
- A route whose auth wiring is **removed** drops out of the auth-wired set and is not exempted, so it FAILS — the literal regression BOB-102 is conditioned on.
- A **stale exemption** for a route that no longer exists FAILS, so the list cannot rot into fiction.
- A **duplicate exemption** for the same `(service, method, path)` FAILS, so a `known-gap` cannot be silently masked by a later `public-by-design` entry and the honest-gap count cannot be skewed (§11.4.261).
- An **unmodelled registration idiom** FAILS, so a route added tomorrow in a form this engine cannot parse cannot slip through unseen.
- A declared LAN service resolving **zero routes** FAILS as BLIND rather than passing quietly, because a blind extractor and a route-free service return the identical quiet zero (§11.4.201(6) FALSE-NULL).

Edge cases

- **Loopback services.** A service declared `lan_bound: false` is out of scope and is not scanned. Firing on it would be a false-positive refusal.

- **Redundant exemptions.** An entry for a route that *is* auth-wired is tolerated (it is harmless) rather than failed, to avoid a false positive on a defensively-listed route.
- **`method: ANY`.** Matches any method for that path — used for stdlib-mux routes, which multiplex methods inside the handler.
- **Aliased dependencies.** A module-level `_g = Depends(require_api_token)` used as `_: None = _g` is recognised as genuine protection, including when imported from a sibling module. Refusing it would be a spurious build failure, and a gate that cries wolf gets bypassed.
- **Malformed policy.** Unparseable YAML exits `2` (ERROR) with the real cause, never `1` with a misattributed "routes do not demand authentication".
- **Dormant markers.** `qbittorrent-proxy-go` currently declares `auth_markers: []`, so every gin route rides an exemption. The per-owner and registration-order logic above is therefore inert for it today and arms itself the moment a gin auth marker is introduced — which is precisely when it starts to matter.
- **Method-scoped keys.** Exemptions are keyed on `(service, method, path)`, so a path exempted for `GET` is **not** exempted when it later gains a `POST`.

Round 9 — what changed and why (independent review remediation)

Round 8 returned NO-GO on the gomux extractor. All three findings were real and are closed here; each is pinned by a fixture that fails against the pre-fix engine (§11.4.115(F)).

BLOCKING — a mutating route was SILENTLY DROPPED. gin refuses a receiver it cannot resolve because `GIN_VERB` is a receiver-agnostic catch-all. gomux had no equivalent: round 7 widened only the *dotted* spelling, so a receiver that was neither a bare identifier nor a dotted selector matched nothing and the skeleton filter stepped over the line with no finding. Measured: `newServeMux().HandleFunc("/wipe", h)` beside a healthy `mux.HandleFunc("/ok", h)` produced `exit 0 / routes 1 / findings 0` with `/wipe` **absent from the inventory** — the gate reported PASS over a wide-open LAN mutating route. `s.buildMux().HandleFunc(...)` and `muxes[0].HandleFunc(...)` dropped the same way. The §11.4.201(6) BLIND net could not catch it: that net fires when a file yields *zero* routes, and the healthy sibling kept the file non-empty — **a drop hides behind a healthy neighbour.** Fixed by `MUX_CALL`, the catch-all gomux lacked, compared *by count* against the receivers this model can name. Counting rather than testing "is any receiver nameable" is what makes a line holding both a nameable and an unnameable

registration refuse instead of half-registering. The invariant: **a `.HandleFunc(/.Handle(` call site this resolver cannot attribute must REFUSE, never skip** (§11.4.252).

> **Round 9 stated that invariant as an absolute the code did not honour, and round 10 falsified it.** `MUX_CALL` requires the dot and the member on ONE line, so `open.` + newline + `HandleFunc("/wipe", h)` — legal, **gofmt-stable** Go — matched *nothing*: not the naming regexes, not the catch-all counter, not the refusal. Both counts were zero, `_sites > _named` was false, and the line fell through to the very skip the invariant forbids. An invariant a resolver cannot honour is not a guarantee, it is a claim; round 11 fixed the **primitive** so the sentence is true, rather than re-stating the sentence. The bound it carries today, stated so the next reader can check it rather than trust it. **A Go registration can legally split in exactly one place — at the selector dot** — and that was established with `gofmt -e`, not assumed: splitting before the `(` is rejected by the parser (`open.HandleFunc` + newline + `("/w", h)` does not compile, because Go inserts a semicolon after the identifier), and splitting *inside* the member name yields two unrelated statements rather than a registration. Every dot-split spelling is therefore covered — same-line whitespace on either side of the dot, a block comment in the gap, a newline, and any number of blank or `//`-comment lines before the member. The neighbouring spellings were measured too and are refused under their own correct labels: a receiver expression split across lines (`s.` + newline + `buildMux().HandleFunc()`) is refused as an unnameable receiver, and arguments moved to the next line are refused as a non-literal path. A method *value* that is not a registration at all (`_ = mux.Handle`) is correctly **not** refused — checked, because a fix that starts firing on that would be the §11.4.201(1) false positive this section is about.

IMPORTANT — a false refusal on idiomatic Go. `FUNC_LIT = \bfunc\s*\(` also matches a *method's receiver clause*, so `_closure_spans` treated an entire method body as a `func(){}` literal and `_function_level_returns` discarded the method's own `return WithAuth(mux)`. Byte-identical bodies gave opposite verdicts: the free-function spelling exited 0, `func (s *Server) Build()` exited 1 claiming the mux was "returned WITHOUT an auth marker". Method-scoped mux construction is idiomatic Go, and a gate that cries wolf on an idiom gets bypassed — which is how the silent-drop class above comes back. The discriminator keys on what *follows* the first balanced paren group: a method continues with its name then `(`, a literal with `{` or a result type. Genuine closures nested inside a method are still closures, pinned separately.

MINOR — an unpinned cycle guard. The `fastapi-import-cycle` fixture builds a cycle in the import graph but never places a verb attribute on a cyclically imported name, so `_router_key` never

recursed into it and deleting the seen-set guard left the fixture green. The new fixture puts `api.post(...)` on a name whose import chain cycles, so the pristine engine terminates and exits 0 while the guard-less mutant raises `RecursionError` — a crash rather than a verdict, which is a §11.4.1 FAIL-bluff.

Correction carried from the review, and one the review got wrong. The skip table's row "line has no mux-registration skeleton … FILTER no route to lose" was false by the table's own taxonomy — the construct *dropped* a route, it did not filter a non-route — and is rewritten to what is measured. Separately, the review recorded the surviving mutation on the gin `len(sites) == 1` credit filter as *redundant* with `unresolvable_groups`; **that reading is wrong.** `GIN_GROUP` carries no leading lookbehind while `GIN_GROUP_DECL` carries one, so a dotted assignment target (`s.g = r.Group("/safe")`) reaches `group_sites` and never `decl_sites`; the "bound N times" refusal is keyed on `decl_sites`, so it never fires. With the filter removed the route resolves as `/safe/wipe` instead of `/wipe`, an operator exemption written for the auth'd `/safe` group matches it, and **the gate goes fully green — exit 0, zero findings — over an unauthenticated mutating LAN route.** The filter is load-bearing, was merely unpinned, and now has its own fixture.

Live posture unchanged. All 13 registrations under the gomux root are nameable receivers, so the real tree still resolves 74 routes across 3 services with 23 honest gaps and the `--json` inventory is byte-identical to round 8.

Round 11 — closing the round-10 NO-GO

Round 10 returned NO-GO with 1 BLOCKING, 2 IMPORTANT, 1 MINOR and 2 NITs. All six are closed here. Two of them were **one primitive wearing two faces**, and the audit that followed found more doors than the report did.

BLOCKING — the gomux twin of a gin fix made five rounds earlier. A Go selector may be separated from its member by arbitrary whitespace, including a newline. Round 6 met this in gin and fixed **gin only**; every gomux regex still required dot-and-member on one line, so `open.` + newline + `HandleFunc("/wipe", h)` matched *nothing* — not the naming regexes, not the round-9 catch-all counter, not the refusal — and the line fell through to the skip. Beside a healthy wrapped sibling: `exit 0 / routes 1 / findings 0 / unmodelled 0`, `/wipe` absent. `gofmt` returns that file **byte-identical**, so a formatter will not save you. Fixed by

extracting the primitive into a shared factory (`_selector_head` / `_ws_selector` / `_selector_is_split`) that both extractors call, so the two halves can no longer drift apart.

IMPORTANT — the same primitive on one line, in *both* extractors: `open.  HandleFunc(`, `r.  POST(`, `mux . Handle(`. The gin case was the worst of the pair — with `r.Use(AuthMW())` above it and one healthy sibling route the gate returned a clean `exit 0 / zero findings` over an unauthenticated mutating LAN route. **The decision, since the round asked for it explicitly: refuse, never resolve.** Refusing is what the shipped gin code already did for the newline half, so refusing the whitespace half makes one primitive behave one way; resolving would mean widening the *credit-granting* regexes — the ones carrying the `(?<![\w.])` carrier lookbehinds and the mutual exclusivity the `_sites/_named` arithmetic rests on — and re-proving all of it, for spellings that do not survive `gofmt` anyway. Only new, separately-labelled *detectors* were added; nothing that grants credit moved.

The door the report did not open. The same primitive also reaches `.Use(`, `gin.Default()`, `http.NewServeMux()` and `.Group(`. `.Group(` had to be fixed because it corrupts the route **path**: a split `.Group("/admin")` makes `POST /admin/wipe` resolve as `POST /wipe`, and an exemption written for a genuinely public `POST /wipe` matches it — `exit 0`, zero findings, over an unauthenticated `/admin/wipe`.

> **This paragraph used to claim the other three "merely drop the credit … loud and fail-closed". For `gin.Default()` that was FALSE, and round 12 found it. See Round 13 — an engine declaration carries no credit to drop, so its absence disarms a refusal instead.**

IMPORTANT — a func TYPE owns no body brace. `FUNC_LIT` matches `func(`, which in Go introduces a *type* at least as often as a *literal*, and a type has no body — so `text.find("{", j)` walked forward and anchored the span on whoever owned the next brace. Round 9 point-fixed one shape (the method receiver); round 10 reported two more; the audit found **eleven**, including four *body* positions (`var mw func(...)`, `type MW func(...)`, `make(chan func(int))`, `struct{ F func(int) int }`) that hijack the brace of a following `if`/`for` block and hide a real `return WithAuth(mux)` inside it — and two *receiverless* shapes, so the defect never needed a method at all. Fixed by two rules that provably do not subsume each other: **RULE S**, nothing before a block's own body brace is a literal, and **RULE D**, a depth-0 `) ] } , ; =` or newline between a parameter list and the candidate `{` proves that brace belongs to someone else. RULE D alone cannot settle `func routes() func(int) int {`, because `func(int) int { return 0 }` **is** a legal literal (verified with `gofmt -e`); RULE S alone cannot see any body shape. Each **rule** is pinned by its own fixture and killed by its own mutation — but round 12 (NIT-2) found that sentence being read as covering the

shapes too, and three of them (free-function func-RESULT, `make(chan func(int))`, `struct{ F func(int) int }`) had no fixture at all. All three probed correct; round 13 pins them so the sentence is true of both readings. The prose above also said "**eleven**" over a list of ten — the method receiver clause, the shape round 9 fixed, was narrated but never enumerated. It is now listed, and the count and the list agree.

MINOR — an unpinned belt, labelled rather than deleted. The `MUX_DOTTED_OWNER` term in `_named` is dead by preemption: the dotted refusal above it returns on `.search()`, which is truthy for every line `.findall()` would count. Verified independently, and the deletion mutation survives the whole matrix — it is *semantically unreachable*, so no fixture can pin it. It is kept as correct arithmetic for a future reordering and recorded in-source as UNPINNED.

The mutual exclusivity it rests on was re-measured over 20,000 adversarial lines with a control needle proving the instrument was seeing — but round 12 found that measurement was a **session artefact**: the in-source note cited "the harness's `r11-mux-exclusivity-fuzz` needle", `grep -c fuzz` on the harness returned **0**, and the mutation that widens `MUX_ANY` (precisely the credit-side widening the *refuse, never resolve* decision exists to prevent) survived all 156 assertions. Round 13 lands the needle for real: 25,000 generated lines from a 34-atom grammar, asserting `named <= sites`, with **two** control needles — the lookbehind-stripped regex and the refused widening itself — both required to report violations before the shipped zero is accepted as evidence (§11.4.201(7)(b)).

Two things this round found that round 10 did not report. The first cut of the BLOCKING fix still dropped a split with a **comment or blank line in the gap** — legal Go, same shape, one gap wider — because it looked only at the immediately following line; the scan now walks forward to the first line carrying code. The second is a fixture defect: two of this round's own new IMPORTANT-2 fixtures carried a *second*, unguarded `return WithAuth(mux)` at function level, so they passed with or without the fix and pinned nothing. The mutation that survived is what exposed them — the md5 delta proved the mutation had applied, so the fault could only be the fixture. Both were rewritten so the only wrap-proving return sits inside the hijacked block, and the mutation then killed them.

Round 13 — the inverted primitive

Round 12 returned NO-GO with 1 BLOCKING, 2 IMPORTANT, 4 MINORs and 2 NITs, and its reviewer wrote eight mutations the author had not — **four survived**. All nine findings are closed here.

BLOCKING — silence is safe for a credit-carrier and unsafe for a refusal-armer

Round 11's honest-scope note claimed `.Use(`, `gin.Default()` and `http.NewServeMux()` alike "each **drop the credit** and are therefore fail-closed and loud". Re-measured one construct at a time, that is true of two of the three and **false of the engine declaration** — and the audit that wrote the sentence never ran the case.

An engine declaration **carries no credit to drop**. `GIN_ENGINE` feeds only the `len(engines) > 1` refusal and the "bound N times" refusal; `GIN_USE` grants credit *independently*, file-scoped, keyed on the bare name. So making the declaration invisible drops nothing — it **disarms the refusal** that is the load-bearing precondition for file-scoped crediting, which is exactly the property the shipped `gin-one-engine-two-funcs` fixture pins.

```
func buildOps() {
    r := gin.New()
    r.Use(AuthMW())
    r.GET("/ok", api.OK)
}

func main() {
    r := gin.
        Default()
    r.POST("/wipe", api.WipeHandler)
    r.Run(":7187")
}
```

`gofmt` returns that file **byte-identical**. `main()`'s engine is a fresh `gin.Default()` carrying no middleware; the round-12 gate reported `POST /wipe auth_wired: true`, resolution *"owner 'r' carries .Use(marker) at or above this line"*, `findings: []`, **exit 0**. Spelled normally, the very same file raises `2 gin engine declarations in one file (r)` and exits 1 — that control is what proves the spelling is the cause.

So the primitive is not "a selector can split at a dot." It is:

> A construct whose **absence drops a credit** degrades fail-closed and loud. A construct whose **absence disarms a refusal** degrades fail-open and silent.

The fix **arms**, it does not refuse separately: an engine *site* feeds two refusal counters and grants nothing, so widening its detector can only make the gate refuse more, never mint a false PASS. The split spelling therefore becomes byte-for-byte the normal spelling instead of growing a second finding class.

The sweep, and what it found

Round 12 named the search: *the remaining constructs whose absence disarms a refusal*. Every refusal-arming construct in both Go extractors was probed — nine constructs, on the whitespace axis, the newline-split axis and the naming axis. **Two more doors were open**, and both are closed here:

- **the aliased import.** `import g "…/gin"` binds the package to `g`, so a hard-coded `gin.` selector is blind to it: `r := g.Default()` beside a `.Use`-carrying sibling engine came back `auth_wired: true`, exit 0, gofmt-**stable**. The local package name is now resolved from the file's own import block. `import . "…/gin"` leaves no selector at all and is refused.
- **the nine `GIN_UNMODELLED` members** (`.Any .Match .Handle .Static .StaticFile .StaticFS .StaticFileFS .NoRoute .NoMetho`). These are *pure* refusal-armers — none ever resolves into a route — so a split spelling armed nothing and the route vanished from the inventory entirely. Measured on all nine, every one gofmt-stable.

Five constructs measured **closed** (the round-11 detectors already catch them): `GIN_DOTTED_OWNER`, `MUX_DOTTED_OWNER`, the `MUX_CALL`/`MUX_ANY` counter, the `http.DefaultServeMux` rule, and the `;`/multi-statement rule. FastAPI is whitespace-immune by construction — it is parsed with `ast`.

The full matrix, the `gofmt` verdict per spelling and the false-positive controls are captured under `docs/qa/BOB-102/round-13/`:

- `door-sweep.md` — the nine refusal-arming constructs, the three axes, and every control
- `gofmt-measurements.txt` — `gofmt -e` legality and byte-stability for all 88 probed spellings
- `mutation-kill-matrix.md` — 18 mutations, each md5-proven applied, with the restore md5-verified
- `MANIFEST` — the reviewed-bytes custody record, and the finding that **all four of this gate's executable artifacts are untracked in git**, which is why no round-over-round diff on this item has ever been checkable

IMPORTANT — a citation to a needle that did not exist

The analyzer cited "the harness's `r11-mux-exclusivity-fuzz` needle" for the `named <= sites` invariant. `grep -c fuzz` on the harness returned **0**, and the mutation widening `MUX_ANY` survived all 156 assertions. The invariant is true; nothing shipped was pinning it. The needle is now real — see the corrected MINOR paragraph in round 11 above.

IMPORTANT — the `.Group(` poison was unpinned, bypassable, and unexplained

Three separate defects in one mechanism:

- **unpinned.** Deleting the poison loop survived all 156 assertions. It is now pinned by a fixture pairing the refused declaration with a *resolvable same-named group in a sibling function* — which is what makes the difference observable, since unpoisoned the route inherits that group's prefix **and** its credit.
- **bypassable.** A Go assignment may break after its `:=`, putting the target one line *above* the trailing dot the refusal fires on. `GO_ASSIGN_TARGET` read only the dot line, so the refusal fired and the poison missed: the gate exited 1 while the inventory reported `POST /safe/wipe auth_wired: true`. The walk-back is one step and gated on the previous code line ending in an assignment operator; over-poisoning is the fail-closed direction.
- **unexplained.** A poisoned name resolves at an *unprefixed* path with no inherited middleware, and the route said only "no `.Use(marker)` covers owner 'g'" — true, and silent about the load-bearing half. The resolution string now states that the prefix was dropped and why. **Measured correction to the report:** the "collateral" (a poisoned name stripping another function's resolvable same-name group) is **not** new to the poison — the ordinary spelling `g := q.Group("/other")` behaves identically, because a name bound twice in a file is refused by the shipped `decl_sites` rule. The defect was the missing label, not the scoping.

MINOR — a trailing dot is not always a selector dot

`x := 3.` is a **float literal**, and Go's semicolon insertion ends the statement after it, so the next line cannot be the far half of a selector. The forward walk refused it — a false refusal on legal, gofmt-stable Go, and by round 9's own doctrine a gate that cries wolf gets bypassed. The discriminator is exact rather than heuristic: a Go identifier may never *begin* with a digit, so a `\w` run ending at the dot whose first character is a digit cannot be a selector operand. `x2.` and `foo[3].` stay selectors; `3.` and `1_000.` are numbers.

MINOR — RULE D's newline escape, and a twelfth shape

The newline escape was load-bearing and unpinned: both shipped body fixtures were rescued by an interposed `mux := …` line whose `=` escaped for them. The new fixture places the func *type* directly before the `if` whose brace it would hijack, with the only wrap-proving return inside it.

And `:` joins the escape set. A depth-0 colon separates a **clause head** from what follows it, so a `{` after one is never the body of a `func(` inside that head — `case func(int) int: if d.On { return WithAuth(mux) }` anchored a phantom body on the `if` brace and refused legal Go. `:=` was already escaping on its `=`, so nothing previously resolvable was widened.

MINOR — cleanup that survives SIGKILL

`trap cleanup EXIT` cannot run under `SIGKILL`; a run killed by the tool-call ceiling leaked 32 GB of `lanauth-meta-*`. The harness now sweeps stale trees at **start**, age-gated at 120 minutes and excluding the current `$TMPROOT` by name, so a concurrently running harness can never have its live directory reaped. Sweep failures are swallowed: housekeeping must never fail the gate.

Verified correct and left alone

The reviewer's four killed mutations all stay killed: forward-walk-over-blanks, RULE S, the ws/split refusal ordering (held by its own label fixture), and the `_body_brace` rewrite. `MUX_DOTTED_OWNER`'s dead term still survives its deletion mutation and keeps its honest **UNPINNED** label — it is semantically unreachable, so no fixture can pin it.

Honest boundary (§11.4.6)

This gate asserts **static route wiring**: that each LAN-reachable route is attached to an auth mechanism. It does **not** assert that the mechanism is armed at runtime. The merge-service token gate is env-conditional — with `BOBA_API_TOKEN` unset, `require_api_token` returns immediately and the route is open in practice. That is a boot-time invariant (§11.4.254), not a static one, and belongs to a separate runtime check. Claiming otherwise here would be the bluff this guard exists to prevent.

The operator has since decided to arm `BOBA_API_TOKEN` and back it with a boot-time invariant; that work is tracked as **BOB-197**. This gate is the static half of the pair and does not supersede it.

Likewise the gate proves coverage of the routes it can resolve; it does not prove the auth mechanisms themselves are correctly implemented.

Related

- Policy/exemption data: `config/lan_route_auth_policy.yaml`
- Analyzer engine: `scripts/pre_build/lan_route_auth_analyzer.py`
- Paired §1.1 meta-test: `tests/pre_build/test_check_cm_lan_routes_authenticated.sh`
- Sibling gates: `check_cm_healthcheck_covers_served_ports.sh`, `check_cm_killpg_pgid_guard.sh`